MFE FORMWORK MR11 SYSTEM

Code Guide: Backend

Version 1.0

Prepared for

MFE Formwork

Prepared by

MyIntellX project team

Date

21 October 2026

Status

Draft for client review

Document control

Item	Detail
Document	Code Guide: Backend
System	MFE Formwork MR11 System, version 1.0
Document version	1.0
Date	21 October 2026
Status	Draft for client review
Based on	Repository izzatmyintellx05/MFE, branch main, commit 0cc8c36
Prepared for	MFE Formwork
Prepared by	MyIntellX project team
Intended readers	Developers taking over the code, client IT
Classification	Confidential, for client and vendor use only

Revision history

Version	Date	Description	Author
1.0	21 October 2026	First issue for client review	MyIntellX project team

Contents

1. Introduction 4
2. Before you start 4
3. How the backend fits together 5
3.1 What happens to a request 5
3.2 Where things live 5
4. Entry points 6
4.1 server.ts, at the top of the repository 6
4.2 server/vercel.ts 6
4.3 server/server.ts, which is no longer used 6
4.4 api/index.js, which is generated 7
5. The Express application (server/app.ts) 7
6. Loading an instance's data (server/bootstrap.ts) 7
7. Middleware 8
7.1 requireAuth (server/middleware/auth.middleware.ts) 8
7.2 requireRoles (server/middleware/rbac.middleware.ts) 8

8. The auth module 8
8.1 auth.routes.ts 8
8.2 auth.controller.ts 8
9. The departments module 9
9.1 department.routes.ts 9
9.2 department.controller.ts 9
9.3 department.service.ts 10
10. The mr11 module 10
10.1 mr11.routes.ts 10
10.2 mr11.controller.ts 10
10.3 The consolidation engine (mr11.engine.ts) 11
10.4 Pricing (mr11.pricing.ts) 13
11. The admin module 14
12. The visualization module 14
13. The data layer 14
13.1 The in-memory store (server/db/prisma.ts) 14
13.2 Talking to PostgreSQL (server/db/supabase.ts) 15
13.3 server/db/prisma.js 16
14. Utilities 16
14.1 Reading Excel files (server/utils/excel-normalizer.ts) 16
14.2 Exchange rates (server/utils/fx.ts) 16
14.3 The LME aluminium price (server/utils/lme.ts) 16
14.4 File storage (server/utils/storage.ts) 17
15. Configuration 17
15.1 The column map (server/config/mr11.config.ts) 17
15.2 Database settings (server/config/database.config.ts) 17
16. Debugging tips 17

1. Introduction

This guide walks through every backend file in the repository and explains what it is for, which functions matter, how they work and what they connect to. It is written for a developer who knows TypeScript and Node.js but hasn't seen this code before.

We suggest reading section 3 first to get the big picture, and then turning to whichever file you need. The code excerpts are copied from commit `0cc8c36`, so they match the release exactly. The frontend and the project's configuration files have guides of their own (Code Guide: Frontend, and Code Guide: Configuration, Build and Scripts).

2. Before you start

Item	Detail
Language	TypeScript as ES modules (`"type": "module"`), run with `tsx` during development
Framework	Express 4
Database	Supabase PostgreSQL, reached through `pg`; there is no ORM when the system runs
Running it	`npm install`, then `npm run dev`, then open http://localhost:3000
Demonstration data	Without `DATABASE_URL`, the server loads the sample workbooks in `uploads/` and keeps everything in memory
Where to start reading	`server/app.ts`, then `server/modules/mr11/mr11.controller.ts`, then the engine

There is one thing worth knowing before you open any file. The code imports `PrismaClient` from `@prisma/client`, but `tsconfig.json` points that name at `server/db/prisma.ts`, a small in-memory database written for this project. No generated Prisma client is used when the system runs, and all lasting data goes to Supabase through `server/db/supabase.ts`.

```
// tsconfig.json
"paths": {
  "@prisma/client": ["./server/db/prisma.ts"]
}
```

3. How the backend fits together

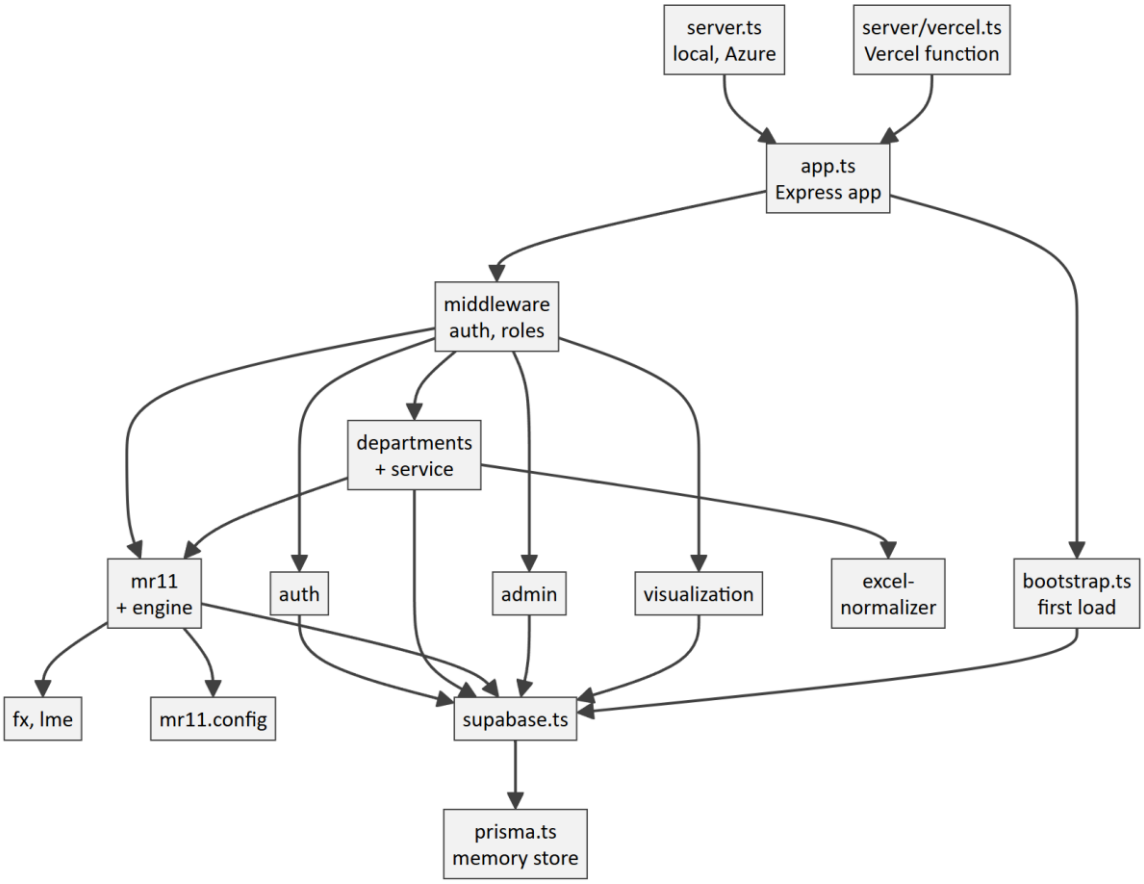


Figure 1: The backend code map, showing the entry points, middleware, modules and data layer

3.1 What happens to a request

1. `server.ts` (or `server/vercel.ts` on Vercel) starts the Express app defined in `server/app.ts`.
2. The app reads JSON bodies of up to 100 MB and answers `/health` straight away.
3. Every other request waits for `ensureBootstrapped()`, which loads users, workbooks and the latest MR11 from Supabase the first time it runs.
4. The router for the path takes over: `/api/auth`, `/api/departments`, `/api/mr11`, `/api/admin` or `/api/visualization`.
5. Middleware checks the token (`requireAuth`) and then the roles (`requireRoles`, or the module's own check).
6. The controller does the work, using the in-memory store and Supabase, and replies with either `{ success, data }` or `{ success: false, error }`.

3.2 Where things live

Folder	What it holds
`server/`	`app.ts`, `bootstrap.ts`, `vercel.ts`, and `server.ts`, which is no longer used

Folder	What it holds
server/middleware/	The token check and the role check
server/modules/<area>/	One folder for each area of the API. Each *.routes.ts file declares its paths and who may call them, and each *.controller.ts file handles the requests
server/modules/mr11/	Also holds mr11.engine.ts, which builds the MR11, and mr11.pricing.ts
server/db/	The in-memory store and the Supabase access code
server/utils/	Reading Excel, exchange rates, the LME price and file storage
server/config/	The MR11 column mapping and the database settings

4. Entry points

4.1 server.ts, at the top of the repository

This file starts the server for local development, Docker and Azure. It works through these steps:

1. It loads .env with dotenv, and sets a fallback JWT_SECRET if none is set. The Operations and Support Guide explains why that fallback must not be relied on in production.
2. It reads the port from PORT or from a --port argument, and uses 3000 if neither is given.
3. In development, or whenever dist/ is missing, it adds Vite's development middleware so that the React app and the API share one port.
4. In production it serves dist/ and sends any unknown path to index.html, so that client-side addresses such as /mr11 still work when the page is refreshed.
5. It starts listening on 0.0.0.0 and then begins ensureBootstrapped() in the background.

```
if (!isProduction || !fs.existsSync(distDir)) {
  const { createServer: createViteServer } = await import('vite');
  const vite = await createViteServer({ server: { middlewareMode: true, ... }, appType: 'spa'
});
  app.use(vite.middlewares);
} else {
  app.use(express.static(distDir));
  app.get('*', (req, res) => res.sendFile(path.join(distDir, 'index.html')));
}
```

4.2 server/vercel.ts

This is the entry point used on Vercel. It exports the same Express app as its default export and starts loading data immediately. npm run build bundles it with esbuild into api/index.js, which Vercel runs as a single serverless function, and vercel.json sends /api/* and /health to it.

4.3 server/server.ts, which is no longer used

An older entry point that refuses to start without JWT_SECRET. Nothing runs it any more; package.json and the Dockerfile both use the server.ts at the top of the repository.

4.4 api/index.js, which is generated

The bundled output of `server/vercel.ts`. Please don't edit it by hand, because `npm run build` writes it again every time.

5. The Express application (server/app.ts)

This file creates the app, adds the common middleware and the health checks, and connects the five routers.

Part	What it does
`cors({ origin: true, credentials: true })`	Accepts requests from any origin, which should be restricted in production
`express.json({ limit: '100mb' })`	Reads JSON bodies of up to 100 MB
`GET /health`	Answers `{ status: 'ok' }` without touching the database
The start-up gate	`app.use((req, res, next) => ensureBootstrapped().then(() => next(), next))` makes every later request wait for the first load
`GET /api/health/db`	Connects to Supabase, lists the tables and returns `describeDatabase()`. It is public, so it should be restricted in production
`resolveRouter()`	Accepts a router whether it was exported as the default, by name or as the module itself, and returns a router that answers 500 if none is found
The routes it connects	`/api/auth`, `/api/departments`, `/api/mr11`, `/api/admin` and `/api/visualization`
Static web files	If `WEB_DIR` (or `../../public`) contains `index.html`, it serves that folder, with a fallback for client-side addresses

6. Loading an instance's data (server/bootstrap.ts)

A serverless instance starts with nothing in memory. This file loads the data once, and makes requests wait until it has finished.

Function	What it does
`ensureBootstrapped()`	Runs `bootstrapSystem()` once and shares its promise. If loading fails, it forgets the promise so that the next request tries again
`bootstrapSystem()`	Loads the users, the active workbooks and the latest MR11. It returns false when a database is configured but couldn't be read

`bootstrapSystem()` works through four steps:

1. `hydrateUsersFromDb()` replaces the built-in demonstration accounts with the users and roles in the database.
2. `hydrateActiveVersionsFromDb()` loads each department's active workbook.

3. Only when there is no database, it looks in `uploads_storage/` and `uploads/` for files ending in `bd.xlsx`, `finance.xlsx` and so on, and loads the newest of each with `processAtomicWorkbookUpload(..., { persist: false, regenerate: false })`. Dispatch's Local and Overseas sample files are found by their names.
4. It takes the latest Mr11Run from Supabase and restores the engine's history from it. If there isn't one, it runs the pipeline in memory instead.

7. Middleware

7.1 requireAuth (server/middleware/auth.middleware.ts)

This reads the `Authorization: Bearer <token>` header, checks the token with `JWT_SECRET`, and puts the decoded `{ id, email, roles }` on `req.user`. A missing token gets 401 `AUTH_REQUIRED`, and a bad or expired one gets 401 `INVALID_TOKEN`.

```
const token = authHeader.split(' ')[1];
const payload = jwt.verify(token, secret) as AuthenticatedUser;
req.user = payload;
next();
```

Because the roles come from the token rather than the database, a change to someone's roles only applies once they sign in again.

7.2 requireRoles (server/middleware/rbac.middleware.ts)

`requireRoles(...roles)` returns middleware that always lets ADMIN through, and lets anyone else through only if one of their roles is on the list. Everyone else gets 403 `ACCESS_DENIED`. It protects `/api/visualization` (ADMIN and CEO) and the MR11 row delete and restore routes (ADMIN only).

8. The auth module

8.1 auth.routes.ts

Route	Handler	Who can call it
POST /login	`login`	Anyone
GET /me	`getMe`	It checks the token itself
POST /change-password	`changePassword`	Signed-in users (`requireAuth`)

8.2 auth.controller.ts

Function	What it does
`hashPassword(password)`	Hashes with PBKDF2-SHA512, 1,000 iterations and a random 16-byte salt, and stores the result as `pbkdf2$<salt>$<hash>`
`verifyPassword(password, stored)`	Accepts the pbkdf2 format, an older `salt:hash` format, bcrypt (`$2...`, used for accounts created in Admin Permissions), unsalted SHA-256 and plain text

Function	What it does
`login`	Checks the input, reloads the users from Supabase, finds the user, checks the password, signs a 7-day JWT containing the id, email, full name and roles, and reports whether the default password is still in use
`getMe`	Checks the token, finds the user (reloading the users once if necessary) and returns their roles
`changePassword`	Requires at least 8 characters, different from the current password and from `admin123`. It saves the new hash to Supabase, and if that save fails it reloads the users so the old password stays in force
`register` and `getCurrentUser`	Not connected to any route; they are left over from an earlier version

```
if (!user) return res.status(401).json({ ... code: 'USER_NOT_FOUND' ... });
if (!verifyPassword(password, user.passwordHash))
  return res.status(401).json({ ... code: 'WRONG_PASSWORD' ... });
const token = jwt.sign({ id: user.id, email: user.email, fullName: user.fullName, roles:
roleCodes },
  getJwtSecret(), { expiresIn: '7d' });
```

The main account, `admin@mfeformwork.com`, always receives the ADMIN role, and if it is missing it is created again with the default password.

9. The departments module

9.1 department.routes.ts

Route	Handler	Notes
GET /	`listDepartments`	Any signed-in user
GET /:code, /:code/workbook and /:code/active	`getActiveDepartmentWorkbook`	That department, or ADMIN
POST /:code/upload and POST /:code	`uploadDepartmentWorkbook`	Multer keeps the file in memory, with a 25 MB limit and the field name `file`
GET /:code/download	`downloadOriginalFile`	Only works for files stored on disk, which means the sample files

`requireDepartmentAccess` lets in ADMIN or the department's own role. SHELLPLAN and DESIGN can open each other's page, because they share one workbook.

9.2 department.controller.ts

Function	What it does
`combinedDepartment(code)`	Treats SHELLPLAN as DESIGN, which is where the combined workbook is kept
`getActiveDepartmentWorkbook`	Reads the active version from Supabase first, falls back to memory, and creates the department row if it is missing
`uploadDepartmentWorkbook`	Checks the role again, reads `?part=` for Dispatch, calls `processAtomicWorkbookUpload`, and reports success or `UPLOAD_FAILED`

Function	What it does
downloadOriginalFile	Sends the stored file if storageKey is a real path. Uploads made in the browser (buffer://...) get 404
listDepartments	Lists the departments from memory, taking each active file's name and time from Supabase

9.3 department.service.ts

This file stores an upload as the department's active version and then rebuilds the MR11. The work happens in processAtomicWorkbookUpload(prisma, deptCode, fileOrBuffer, name, mime, size, userId, options), which goes through six steps:

1. It reads the workbook with parseAndNormalizeWorkbook().
2. It finds the department and a valid uploader id, falling back to the main admin account.
3. For Dispatch only, it decides whether the file is Local or Overseas (from options.dispatchPart, or otherwise with detectDispatchPart(), which looks at the file name and then at the ETD and ATD headers) and combines it with the other file using combineDispatchFiles().
4. It creates the FileVersion in memory and then saves it to Supabase with saveFileVersionToDb(). If the save fails, it throws an error so that the user is told.
5. Only after a successful save does it make this the department's active version and write an UPLOAD_WORKBOOK entry to the audit log.
6. It reloads the other departments' latest versions from Supabase and runs executeMr11Pipeline().

options.persist and options.regenerate are both false when the start-up loader is loading the sample files.

The file also contains parseAndMapShellPlanRows, parseAndMapDesignRows, aggregateShellPlanForMR11 and aggregateDesignForMR11. Nothing calls them any more, because the engine now does that work.

10. The mr11 module

10.1 mr11.routes.ts

Every route needs a signed-in user. GET /, POST /regenerate, GET /export, GET /planning-series, GET /production-series, GET /shellplan-history and GET /dispatch-monthly are open to every role. GET /deleted-rows, POST /rows/delete and POST /rows/restore need ADMIN.

10.2 mr11.controller.ts

Function	What it does
getLatestMr11	Fetches the latest run (from Supabase first) and today's exchange rates and LME price, runs applyLivePricing() on the rows, and adds the column details from mr11.config.ts
triggerMr11Regenerate	Reloads the active versions from Supabase and runs the pipeline; this is what the Sync button calls

Function	What it does
`getShellplanApprovalHistory`	Turns `sourceSnapshot.shellplanApprovalHistory` into a list of rows, newest first
`getDispatchMonthly`	Returns the rows that have `_dispatchedByMonth` figures, for the Dispatch page
`getDeletedRows`, `deleteMr11Row` and `restoreMr11Row`	Read or change `sourceSnapshot.deletedRows`, then rebuild the MR11 with the new list
`getPlanningSeriesHistory` and `getProductionSeriesHistory`	Return the series history from memory, sorted by project, stream and series
`exportMr11ToExcel`	Builds the .xlsx file with ExcelJS, as described below

The export is built in six steps:

1. The rows are priced for today, just as they are on screen.
2. Two header rows are written. Grouped columns share a merged top cell, and the month columns add ACTUAL or F'CAST.
3. Each row is written. Columns with a fixed number of decimal places are written as numbers so that Excel formats them, while identifier columns keep their digits exactly.
4. Each row is given the font and fill colour of its BD row.
5. The design completion dates are written as coloured text (green, yellow or red).
6. The Shell Plan and Design cells are merged over each block, using `_streamSpan`.

10.3 The consolidation engine (mr11.engine.ts)

The engine builds the MR11 from every department's active workbook. At about 2,200 lines it is the largest file in the project, so this section is meant as a map of it.

The helper functions near the top of the file are:

Function	What it does
`parseNumeric`	Reads a number from a cell, or 0 when there isn't one
`cleanStr`	Keeps only lower-case letters and digits, for comparing names and keys, so that "AP 24" and "ap24" match
`normalizeStream`	Turns "Stream 2" and "2.0" into "2". An empty stream becomes "1"
`normalizeColor` and `normalizeFillColor`	Turn colours into #RRGGBB, with an empty font treated as black. White, black, transparent and empty fills all count as no fill
`isBlueColor`	Recognises the blue font of Re-Design Only rows, meaning a clearly coloured hue between 185° and 250°
`colourDistance(a, b)`	The straight-line distance between two colours in RGB, from 0 up to about 441
`closestColourGroup(rows, font, fill)`	Finds the rows whose font and fill are closest, within `SAME_COLOUR_DISTANCE` (100)
`closestFillGroup(rows, fill)`	Does the same using the fill alone, for files that mark their blocks only by fill

Function	What it does
`splitProjectCode(code)`	Splits 251242 into project 25124 and stream 2, and 2512410 into stream 10
`mr11RowKey(...)`	Builds the identity that recognises a row from one build to the next
`malaysiaDate()`	Today's date in Asia/Kuala_Lumpur, as YYYY-MM-DD
`parseFlexibleDate` and `formatDateString`	Turn Excel serial numbers, Date objects and text dates into YYYY-MM-DD
`resolveConsolidated*Status`	Combines the Design, holing, accessories or Shell Plan statuses of several rows into one
`refineHeaders`	Brings up the sub-headers under a repeated group header, and renames duplicate headers
`sheetToRecordsWithStyles(sheet)`	Turns a sheet into extracted rows with their colours (the Data Models document describes these under "Extracted rows")
`buildFinanceMonthColumns(headers)`	Finds BD's monthly ACTUAL and F'CAST columns
`findCellValue(row, header)`	Reads a value by its header, ignoring capital letters and trailing spaces
`applyMonthlyForecast(rows, month, history)`	Fills in Jan-26 to Dec-27 and the yearly totals, along with each row's `_monthStages`

```
const SAME_COLOUR_DISTANCE = 100;
function colourDistance(a: string, b: string): number {
  const rgb = (h: string) => [1, 3, 5].map((i) => parseInt(h.slice(i, i + 2), 16) || 0);
  const [x, y] = [rgb(a), rgb(b)];
  return Math.sqrt(x.reduce((sum, v, i) => sum + (v - y[i]) ** 2, 0));
}
```

The main function, `executeMr11Pipeline(prisma, { persist, deletedRows })`, works through thirteen steps:

1. **History.** It reads the latest Mr11Run from Supabase and restores the series history from it, so that each build carries on from the last one, even on a brand-new instance.
2. **Market data.** It fetches today's LME price and USD exchange rates.
3. **Reading the workbooks.** For each department's active version, it runs `sheetToRecordsWithStyles()` on the first sheet. Dispatch reads its Local and Overseas parts separately and labels each row with the file it came from.
4. **Shell Plan & Design.** If the Design file contains the Shell Plan columns, its rows are used as the Shell Plan rows too. BD rows that have no values at all are dropped.
5. **Planning series.** For every Planning row with a series number, it updates the series history (taking the project number from BD by short name) and adds up the quantities for each project, stream and colour.
6. **Production series.** It does the same for Production, finding Cumulative Produced by its header name and only falling back to column Q if it can't.
7. **The previous run.** It loads the previous rows (so that dates stay put while a total doesn't change), the monthly dispatch and production history, the month-stage history and the approval history.

8. **Each BD row.** It copies the BD columns (matching the whole header, with aliases allowed), keeps the price inputs in `_pricing`, and then brings in the other departments:
    - Finance, matched by the same project and stream, then the closest colour, then the product type.
    - Dispatch. It picks the Local or Overseas file, takes the closest colour blocks, adds up Cumulative Dispatched, and builds the ETD/ATD lines, the Dispatch Date and the m² for each month.
    - Design and Shell Plan, using the fill-colour block for the same project and stream. It records the status, the completion dates for each level, and any new approvals in the history.
    - Planning. Total Processed comes from Cumulative Processed (Project). Processed Date becomes today if the total changed, and otherwise keeps the date from before.
    - Production, giving Total Produced and the latest daily column with output. Blue-font rows show "-" for Production and Dispatch.
9. **Monthly columns.** `applyMonthlyForecast()` fills in the month columns and the CEO month stages.
10. **Deleted rows.** Rows whose `_rowKey` is on the deleted list are left out.
11. **Design status for each project.** Fully Complete, Partially Complete, Ongoing or To Start, for each fill-colour block.
12. **Sorting and merging.** Rows are sorted by project, stream, unfilled before filled, fill colour, and then font colour with black first. `_streamSpan` is set on the first row of each block.
13. **Saving.** The run is stored in memory and, when `persist` is on, in Supabase, with the whole history in `sourceSnapshot`. `saveMr11RunToDb()` skips the save if any department has a newer upload than the one this build used.

10.4 Pricing (mr11.pricing.ts)

Function	What it does
`currencyCode(value)`	Turns "SGD", "Singapore Dollar", "S$", "RM" or "SGD 200" into an ISO currency code, using a table of symbols and the 162 currency names provided by `Intl.DisplayNames`
`priceBdRow(inputs, lmeCash, rates)`	Works out the Selling Price in USD, the LME Adjusted figure and the Final Selling Price for one row
`applyLivePricing(records, lmeCash, rates)`	Runs `priceBdRow` on every row that has `_pricing`

```
if (type === 'freeze') lmeAdjusted = 'Check';
else if (type === 'fixed') lmeAdjusted = 0;
else if (type === 'variable' || type.includes('lme')) {
  const rate = num(p.lmeRate);
  lmeAdjusted = rate === null ? 0 : lmeCash === null ? 'Check'
    : round6(((lmeCash - rate) / 1000) * LME_USD_PER_1000);
}
```

If a price is in a currency that can't be converted, it is left out of the total and explained in `_priceNote`, which the MR11 page shows against the Selling Price cell.

11. The admin module

admin.routes.ts has its own authenticateAdmin check. It verifies the token and then allows ADMIN, an older departmentRole of ADMIN, or the email address admin@mfeformwork.com. The routes are registered more than once in this file. The duplicates do no harm, but they can be removed.

Function (in admin.controller.ts)	What it does
DEFAULT_PROFILE_TABS	The roles that come with each preset. ADMIN gets ADMIN plus every department, and each department preset gets that department's role. There is no CEO preset
getAllUsersWithPermissions	Reloads the users and lists them with their role codes
updateUserPermissions	Replaces a user's roles from a preset or a list, makes sure the main admin keeps ADMIN, and saves the result to Supabase
createUser	Hashes the password with bcrypt (cost 10), creates the user and their roles, saves them to Supabase, and removes the user again if that save fails
deleteUser	Refuses to delete the main admin, and otherwise deletes from Supabase first and then from memory
persistUser(userId)	Saves one user and their roles, and reloads from the database if the save fails

12. The visualization module

GET /api/visualization (for ADMIN and CEO) adds up the projects, the selling price, and the ordered and produced m², and counts the rows by design status and by stream. It looks for columns called total quantity ordered m2 and produced qty, which don't exist in the MR11, so those two totals always come out as 0. The CEO Executive Hub doesn't use this endpoint.

13. The data layer

13.1 The in-memory store (server/db/prisma.ts)

This is a small in-memory database with a Prisma-style interface, which lets the rest of the code read as if it were using Prisma. Each model offers the methods the code actually needs: findMany and create on most models, findUnique, findFirst, update and upsert on many, and delete, deleteMany, createMany and count where they are used, along with $transaction.

Part	What it does
store	Maps for users, roles, user roles, departments, file versions and the Planning and Production series, plus lists for audit logs and MR11 runs
Seed data	The nine roles (role-admin and the rest), the seven departments (dept-bd and the rest), the main admin and eight demonstration accounts, all with the password admin123
matchesWhere(item, where)	Supports equality, OR, and in, gt, gte, lt and lte, as well as Prisma's compound unique keys such as projectNo_stream_fontColor_seriesNumber

Part	What it does
`populateUser` and `populateDepartment`	Imitate Prisma's `include` and `select` for roles, the active version and the uploader
`$transaction(fn)`	Calls `fn` with the same client. It cannot roll anything back
`replaceUsers(users, userRoles)`	Swaps in the users from the database
`exportEngineHistory()` and `importEngineHistory()`	Save and restore the three series collections inside each Mr11Run
`normalizeDatabaseUrl(url)`	Encodes an `@` that appears inside a database password

13.2 Talking to PostgreSQL (server/db/supabase.ts)

All reads from and writes to Supabase go through this file. Because the repository has two table layouts, reads use `SELECT *`, and writes only include the columns that the live table actually has.

Function	What it does
`isDatabaseConfigured()`	Tells you whether a database address has been set
`withPool(fn)`	Opens a connection pool, runs `fn`, tries once more if the connection drops, and returns null if it fails
`tableColumns(pool, table)`	Reads a table's column names and remembers them
`upsertRow(pool, table, values, updateCols)`	Runs `INSERT ... ON CONFLICT ("id") DO UPDATE`, using only the columns that exist
`inTransaction(fn)`	Wraps the work in BEGIN and COMMIT, and rolls back and throws an error if anything fails
`fetchActiveVersionsFromDb()` and `fetchActiveVersionForDepartment()`	Fetch the active workbook for each department
`fetchLatestMr11RunFromDb()`	Fetches the newest Mr11Run, taking the rows from either `calculatedFields` or `records`
`fetchActiveFileSummariesFromDb()`	Fetches the active file names and times, without the workbook data
`hydrateUsersFromDb()`	Replaces the users and roles in memory with the ones in the database
`saveUserToDb()` and `deleteUserFromDb()`	Save or delete a user and their role links in a single transaction
`restoreEngineHistory(run)`	Loads the series history from a run's snapshot
`saveFileVersionToDb()`	Saves an upload, marks the older versions as no longer the latest, and sets the department's active version
`saveMr11RunToDb()`	Saves a run, unless a department's active version has changed since the run was built
`describeDatabase()`	Gathers the columns, active files and row counts for `/api/health/db`
`hydrateActiveVersionsFromDb(prisma, skip)`	Copies newer active versions from the database into memory, ignoring any that changed after the read began
`getLatestMr11Run(prisma)`	Looks in the database first, then in memory

13.3 server/db/prisma.js

A compiled copy of `prisma.ts`. Nothing imports it, so it can be removed.

14. Utilities

14.1 Reading Excel files (server/utils/excel-normalizer.ts)

`parseAndNormalizeWorkbook(fileOrBuffer)` reads a workbook with ExcelJS and returns one FortuneSheet for each worksheet. The Data Models document describes this structure under "The workbook model".

Helper	What it does
`cellValue(cell)`	Takes a formula's saved result, because ExcelJS leaves a result of 0 out of `value`
`extractRawValue(v)`	Turns rich text into plain text, a formula into its result (or null if no result was saved), dates into YYYY-MM-DD and hyperlinks into their text. Excel errors keep their error text
`readThemeColors`, `applyTint` and `colorHex`	Turn theme colours such as "Blue, Accent 4, lighter 40%" into hex values

Each merge is recorded once in `config.merge`, and every cell inside a merge is given the top-left value, so the engine can read any row of a merged block.

14.2 Exchange rates (server/utils/fx.ts)

Function	What it does
`loadBnm()`	Fetches Bank Negara Malaysia's middle rates and converts them into "1 USD = x"
The fallbacks	open.er-api.com, and then Frankfurter (ECB), for currencies that BNM doesn't publish
`getUsdRatesInfo()`	Combines the sources and keeps the result for an hour. If every source fails, it reuses the last result, and only uses `USD_TO_MYR_RATE` as a last resort
`getUsdRates()` and `getUsdToMyrRate()`	Simpler ways to get at the same data

14.3 The LME aluminium price (server/utils/lme.ts)

`getLmeAluminiumPrice()` downloads Westmetall's LME aluminium table, and `parseWestmetallTable()` reads its two newest rows to get the cash price, the 3-month price, the previous cash price and the date. The result is kept for an hour. If the download fails, the last price is returned with `live: false`.

14.4 File storage (server/utils/storage.ts)

`saveFileToStorage()` and `deleteStorageFile()` work with the `STORAGE_DIR` folder. Version 1.0 doesn't call them, because uploads are kept in memory so that the system can run on serverless hosting.

15. Configuration

15.1 The column map (server/config/mr11.config.ts)

This file holds the column map that decides what goes into the MR11. When a department changes a header in its template, this is usually where you update it.

Export	What it means
`ORDERED_HEADER_LIST`	The 78 MR11 columns, in the order they are shown
`MR11_ORDERED_COLUMNS`	For each column: the target, the source department, the source header, the type, whether the match must be exact, and any aliases
`MR11_HEADER_GROUPS`	The two-row Payment terms group
`MR11_NUMBER_FORMATS` and `MR11_USD_AMOUNT_COLUMNS`	The number of decimal places, and the columns shown with a dollar sign
`MR11_SOURCE_KEY_MAP`	The headers that identify the project in each department's file
`MR11_DEPARTMENT_COLORS` and `MR11_COLUMN_DEPARTMENTS`	The highlight colours, and the department each column comes from

```
{ target: 'Countries', sourceDept: RoleCode.BD, sourceColumn: 'Countries', type: 'string',
  exact: true, aliases: ['Country'] },
```

15.2 Database settings (server/config/database.config.ts)

`getDatabaseUrl()` reads `DATABASE_URL`, `DATABASE_POSTGRES_URL` or `POSTGRES_URL`, and changes `sslmode=require` to `no-verify`, because node-postgres would otherwise reject Supabase's certificate chain. `getSslConfig()` turns SSL off for localhost, and whenever `sslmode=disable` is set.

16. Debugging tips

Log message starts with	Where it comes from	What it means
`[MFE Formwork MR11]`	`server.ts` and `bootstrap.ts`	Start-up, and what was loaded
`[DATABASE] read notice` or `retrying read`	`supabase.ts`	A database read failed, or was tried again
`[DATABASE] MR11 run ... not saved`	`saveMr11RunToDb()`	The build used an older workbook than the one in the database
`[DEPARTMENT SERVICE] Could not save upload`	`department.service.ts`	The database rejected the upload
`[FX] ... unavailable` or `[LME] price unavailable`	`fx.ts` and `lme.ts`	A market-data source couldn't be reached

Log message starts with	Where it comes from	What it means
`Auto MR11 pipeline calculation error`	`department.service.ts`	The rebuild after an upload failed, although the upload itself was saved